

Stack Overflow is a community of 4.7 million programmers, just like you, helping each other.

Join them; it only takes a minute:

Sign up

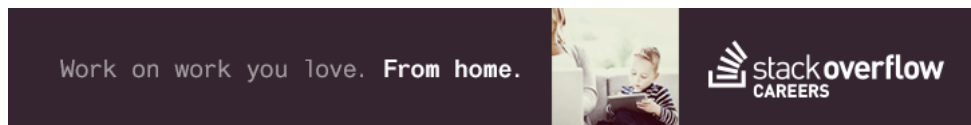
Join the Stack Overflow community to:

Ask programming questions

Answer and help your peers

Get recognized for your expertise

Java Array of unique randomly generated integers



```
public static int[] uniqueRandomElements (int size) {

    int[] a = new int[size];

    for (int i = 0; i < size; i++) {
        a[i] = (int)(Math.random()*10);

        for (int j = 0; j < i; j++) {
            if (a[i] == a[j]) {
                a[j] = (int)(Math.random()*10);
            }
        }
    }

    for (int i = 0; i < a.length; i++) {
        System.out.print(a[i]+" ");
    }
    System.out.println();
    return a;
}
```

I have a method above which should generate an array of random elements that the user specifies. The randomly generated integers should be between 0 and 10 inclusive. I am able to generate random integers but the problem I have is checking for uniqueness. My attempt to check for uniqueness is in my code above but the array still contains duplicates of integers. What am I doing wrong and could someone give me a hint?

java arrays

edited Nov 18 '13 at 2:24



Ray Toal

43.8k 7 76 123

asked Nov 18 '13 at 2:22



AvP

145 1 10

3 What happens when size > 10 ? How can you generate unique numbers for each index? – Jeffrey Nov 18 '13 at 2:23

4 Answers

```
for (int i = 0; i < size; i++) {
    a[i] = (int)(Math.random()*10);

    for (int j = 0; j < i; j++) {
        if (a[i] == a[j]) {
            a[j] = (int)(Math.random()*10); //What's this! Another random number!
        }
    }
}
```

You do find the duplicate values. However, you replace it with another random number that may be a duplicate. Instead, try this:

```
for (int i = 0; i < size; i++) {
    a[i] = (int)(Math.random()*10); //note, this generates numbers from [0,9]

    for (int j = 0; j < i; j++) {
        if (a[i] == a[j]) {
            i--; //if a[i] is a duplicate of a[j], then run the outer loop on i again
            break;
        }
    }
}
```

However, this method is inefficient. I recommend making a list of numbers, then randomizing it:

```
ArrayList<Integer> a = new ArrayList<>(11);
for (int i = 0; i <= 10; i++){ //to generate from 0-10 inclusive.
```

```

        a.add(i);
    }
    Collections.shuffle(a);
    a = a.sublist(0,4);
    //turn into array

```

Or you could do this:

```

ArrayList<Integer> list = new ArrayList<>(11);
for (int i = 0; i <= 10; i++){
    list.add(i);
}
int[] a = new int[size];
for (int count = 0; count < size; count++){
    a[count] = list.remove((int)(Math.random() * list.size()));
}

```

edited Nov 18 '13 at 2:43

answered Nov 18 '13 at 2:27



Justin

2,378 4 16 45

This was a very simple edit to my code! Thanks :). – AvP Nov 18 '13 at 2:30

If you have a duplicate you only regenerate the corresponding number once. But it might create another duplicate. You duplicate checking code should be enclosed in a loop:

```

while (true) {
    boolean need_to_break = true;
    for (int j = 0; j < i; j++) {
        if (a[i] == a[j]) {
            need_to_break = false; // we might get another conflict
            a[j] = (int)(Math.random()*10);
        }
    }
    if (need_to_break) break;
}

```

But make sure that `size` is less than `10`, otherwise you will get an infinite loop.

Edit: while the above method solves the problem, it is not efficient and should not be used for large sized arrays. Also, this doesn't have a guaranteed upper bound on the number of iterations needed to finish.

A better solution (which unfortunately only solves second point) might be to generate a sequence of the distinct numbers you want to generate (the `10` numbers), randomly permute this sequence and then select only the first `size` elements of that sequence and copy them to your array. You'll trade some space for a guarantee on the time bounds.

```

int max_number = 10;
int[] all_numbers = new int[max_number];
for (int i = 0; i < max_number; i++)
    all_numbers[i] = i;

/* randomly permute the sequence */
for (int i = max_number - 1; i >= 0; i--) {
    int j = (int)(Math.random() * i); /* pick a random number up to i */

    /* interchange the last element with the picked-up index */
    int tmp = all_numbers[j];
    all_numbers[j] = all_numbers[i];
    all_numbers[i] = tmp;
}

/* get the a array */
for (int i = 0; i < size; i++)
    a[i] = all_numbers[i];

```

Or, you can create an `ArrayList` with the same numbers and instead of the middle loop you can call `Collections.shuffle()` on it. Then you'd still need the third loop to get elements into `a`.

edited Nov 18 '13 at 2:33

answered Nov 18 '13 at 2:26



Mihai Maruseac

12.7k 5 32 95

I appreciate your answer but I haven't had enough experience with Java to understand your answer. – AvP Nov 18 '13 at 2:31

Ummm, is there a Java compiler that lets you use `break` as a variable name? – ajb Nov 18 '13 at 2:33

Thanks, I didn't code in Java for several years and I forgot that :) – Mihai Maruseac Nov 18 '13 at 2:35

@MihaiMaruseac There aren't many languages that let you use variable names that are the same as keyword names. I'd think `if (break) break;` should have been a warning flag, in any language ... :) – ajb Nov 18 '13 at 5:16

Haskell programmer here :) – Mihai Maruseac Nov 18 '13 at 13:10

It might work out faster to start with a **sequential array** and **shuffle** it. Then they will all be **unique by definition**.

Take a look at [Random shuffling of an array](#), and at the `Collections.shuffle` function.

```
int [] arr = {1,2,3,...(size)}; //this is pseudo code  
Collections.shuffle(arr); // you probably need to convert it to List first
```

answered Nov 18 '13 at 2:29



Karthik T

21.4k 25 50

1 Probably need? You definitely need to convert to a list. – [Justin](#) Nov 18 '13 at 2:42

If you just don't want to pay for the added overhead to `ArrayList`, you can just use an array and use [Knuth shuffle](#):

```
public Integer[] generateUnsortedIntegerArray(int numElements){  
    // Generate an array of integers  
    Integer[] randomInts = new Integer[numElements];  
    for(int i = 0; i < numElements; ++i){  
        randomInts[i] = i;  
    }  
    // Do the Knuth shuffle  
    for(int i = 0; i < numElements; ++i){  
        int randomIndex = (int)Math.floor(Math.random() * (i + 1));  
        Integer temp = randomInts[i];  
        randomInts[i] = randomInts[randomIndex];  
        randomInts[randomIndex] = temp;  
    }  
    return randomInts;  
}
```

The above code produces `numElements` consecutive integers, without duplication in a uniformly random shuffled order.

edited Aug 12 '15 at 5:57

answered Aug 12 '15 at 5:51



balajeerc

1,080 11 24